

Laravel API Best Practices

Expanded Developer Guide with Simple Examples

Structure & Routing & Auth & Validation & Resources & Errors & Security & Performance & Testing &
Versioning

Laravel v7-v12 | PHP 8.0+ | February 2026

Table of Contents

01	Project Structure & Organisation
02	Routing Best Practices
03	Authentication & Authorisation
04	Request Validation
05	API Resources & Response Shaping
06	Error Handling & Exceptions
07	Security Best Practices
08	Performance Optimisation
09	Testing API Endpoints
10	API Versioning
11	Rate Limiting
12	API Documentation
REF	Quick Reference Cheatsheet

CHAPTER 01

Project Structure & Organisation

Keep your API clean, scalable, and maintainable from day one.

The Core Idea

Controllers should only handle HTTP concerns: receive the request, call a service, and return a response. All business logic belongs in Service or Action classes.

Bad — avoid this:

```
// PostController.php — doing too much (avoid) public function store(Request $request) { $post = new Post(); $post->title = $request->title; $post->slug = Str::slug($request->title); $post->user_id = auth()->id(); $post->save(); Notification::send($post->followers, new NewPostNotification($post)); return response()->json($post, 201); }
```

Good — do this instead:

```
// PostController.php — thin, just HTTP public function store(StorePostRequest $request) { $post = $this->postService->create($request->validated()); return new PostResource($post); } // PostService.php — business logic lives here public function create(array $data): Post { $post = Post::create([...$data, 'slug' => Str::slug($data['title'])]); Notification::send($post->followers, new NewPostNotification($post)); return $post; }
```

Recommended Folder Layout

app/ directory structure

```
app/ Http/ Controllers/Api/V1/ # PostController.php, UserController.php Requests/ # Form Request validation classes Resources/ # API Resource transformers Middleware/ # ForceJson, ApiVersion... Models/ # Eloquent models Services/ # PostService, AuthService... Actions/ # Single-responsibility action classes Enums/ # PHP 8.1 enums Policies/ # Authorization policies routes/ api.php # All API routes tests/ Feature/Api/ # Feature tests per endpoint Unit/ # Unit tests for services/helpers
```

Tip: Use single-action controllers (`__invoke` pattern) for complex endpoints — it makes each endpoint's logic instantly obvious and easy to test.

CHAPTER 02

Routing Best Practices

Design clean, predictable, RESTful API routes that are easy to maintain.

Use HTTP Verbs Correctly

Never invent custom action URLs. Use nouns for resources and HTTP verbs for actions. Keep nesting to a maximum of two levels.

Bad — avoid this:

```
// RPC-style – avoid POST /api/getPost POST /api/deletePost POST /api/updateUserPassword
```

Good — do this instead:

```
// RESTful – clear verb + noun GET /api/v1/posts // list all POST /api/v1/posts // create new GET  
/api/v1/posts/{post} // fetch single PATCH /api/v1/posts/{post} // partial update DELETE  
/api/v1/posts/{post} // delete GET /api/v1/posts/{post}/comments // nested resource
```

Route Organisation

routes/api.php

```
Route::prefix('v1')->name('v1.')->group(function () { // Public routes Route::get('posts',  
[PostController::class, 'index']); Route::get('posts/{post}', [PostController::class, 'show']); //  
Protected routes Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {  
Route::apiResource('posts', PostController::class) ->except(['index', 'show']); // Scoped nested:  
ensures comment belongs to post (auto 404 if not) Route::scopeBindings()->group(function () {  
Route::apiResource('posts.comments', PostCommentController::class); }); }); });
```

Tip: Add `getRouteKeyName()` to your model returning 'slug' or 'uuid' to resolve routes by slug/UUID instead of numeric IDs — better for security and readability.

CHAPTER 03

Authentication & Authorisation

Choose the right auth strategy and enforce permissions at every layer.

Sanctum Token Auth

For most Laravel APIs — SPAs, mobile apps, first-party consumers — Sanctum is the recommended solution. Use Passport only for full OAuth2 third-party integrations.

Login endpoint — issue a scoped token

```
Route::post('/login', function (Request $request) { if (!Auth::attempt($request->only('email', 'password'))) { return response()->json(['message' => 'Invalid credentials'], 401); } $user = Auth::user(); // Rotate: delete old token for this device before issuing a new one $user->tokens()->where('name', $request->device)->delete(); // Scoped token – this client can only READ posts, not write them $token = $user->createToken($request->device, ['posts:read']); return response()->json(['token' => $token->plainTextToken]); });
```

Policies for Ownership Checks

Always use Laravel Policies for resource-level authorisation. Never do ownership checks inline in controllers.

PostPolicy.php + Controller

```
// PostPolicy.php class PostPolicy { public function update(User $user, Post $post): bool { return $user->id === $post->user_id; // only the author can edit } public function delete(User $user, Post $post): bool { return $user->is_admin || $user->id === $post->user_id; } } // Controller – one line replaces all manual checks public function update(UpdatePostRequest $request, Post $post) { $this->authorize('update', $post); // auto 403 if policy returns false $post->update($request->validated()); return new PostResource($post->fresh()); }
```

Tip: Always check token abilities in addition to policies. Use `$request->user()->tokenCan('posts:write')` or Sanctum's `CheckAbilities` middleware.

CHAPTER 04

Request Validation

Validate all incoming data with Form Requests — never trust client input.

Form Request Classes

Move all validation logic out of controllers into dedicated Form Request classes. This keeps controllers clean and makes validation reusable and testable.

Bad — avoid this:

```
// Avoid inline validation in controllers public function store(Request $request) {  
    $request->validate(['title' => 'required']); Post::create($request->all()); // mass-assignment risk!  
}
```

Good — do this instead:

```
// StorePostRequest.php class StorePostRequest extends FormRequest { public function authorize():  
bool { return $this->user()->tokenCan('posts:write'); } public function rules(): array { return [  
    'title' => ['required', 'string', 'max:255'], 'body' => ['required', 'string', 'min:10'], 'status'  
=> ['required', Rule::enum(PostStatus::class)], 'category_id' => ['required', 'integer',  
'exists:categories,id'], 'tags' => ['array', 'max:10'], 'tags.*' => ['string', 'max:50'], ]; }  
protected function prepareForValidation(): void { $this->merge(['title' => trim($this->title)]); } }  
// Controller is now one clean line: public function store(StorePostRequest $request) {  
    Post::create($request->validated()); // only safe, validated fields }  
}
```

Tip: Always pass `$request->validated()` to models — never `$request->all()`. This prevents mass-assignment of unrequested fields.

CHAPTER 05

API Resources & Response Shaping

Transform Eloquent models into consistent, versioned JSON responses.

Never Return Raw Models

API Resources give you a stable contract with clients. Raw Eloquent models expose every column including sensitive ones, and make versioning painful.

Bad — avoid this:

```
// Exposes ALL columns, including sensitive internal ones return response()->json(Post::find($id));
```

Good — do this instead:

```
// PostResource.php class PostResource extends JsonResource { public function toArray(Request $request): array { return [ 'id' => $this->id, 'title' => $this->title, 'body' => $this->body, 'status' => $this->status->value, // Only included if eager-loaded – no N+1 'author' => new UserResource($this->whenLoaded('user')), // Admin-only fields – invisible to regular users $this->mergeWhen($request->user()->is_admin, [ 'internal_notes' => $this->notes, ]), 'created_at' => $this->created_at->toISOString(), ]; } } public function show(Post $post) { return new PostResource($post->load('user')); }
```

HTTP Status Code Reference

200 OK	Successful GET/PUT/PATCH	Fetches or updates resource
201 Created	Successful POST	POST /posts — new post created
204 No Content	Successful DELETE	DELETE /posts/1
401 Unauthorized	No or invalid token	Missing Bearer token
403 Forbidden	Authenticated but not authorized	Deleting another user's post
404 Not Found	Resource does not exist	GET /posts/999
422 Unprocessable	Validation failed	Invalid email format
429 Too Many Req.	Rate limit exceeded	throttle middleware hit

CHAPTER 06

Error Handling & Exceptions

Return consistent, informative JSON errors for every failure scenario.

Centralised Exception Handling

Configure all API exception handling in one place. Return a predictable JSON structure for all errors so API clients can handle them uniformly.

bootstrap/app.php (Laravel 11+)

```
->withExceptions(function (Exceptions $exceptions) { // Force JSON for all api/* routes
$exceptions->shouldRenderJsonWhen( fn(Request $r) => $r->is('api/*') || $r->wantsJson() ); // Clean
404 for missing models $exceptions->render(function (ModelNotFoundException $e) { return
response()->json([ 'error' => 'not_found', 'message' => 'The requested resource was not found.', ],
404); }); // Rate limit hit $exceptions->render(function (ThrottleRequestsException $e, Request $r)
{ return response()->json([ 'error' => 'rate_limit_exceeded', 'message' => 'Too many requests.
Please slow down.', 'retry_after' => $e->getHeaders()['Retry-After'] ?? 60, ],
429)->withHeaders($e->getHeaders()); }); })
```

Consistent Error Envelope

Every error — validation, auth, not found, server error — should return the same predictable JSON shape:

Example error responses

```
{ // 422 Validation error "error": "validation_failed", "message": "The given data was invalid.",
"errors": { "email": ["The email field is required."], "password": ["Must be at least 8
characters."] } } { // 401 Unauthenticated "error": "unauthenticated", "message": "Please provide a
valid API token." } { // 403 Forbidden "error": "forbidden", "message": "You do not have permission
to perform this action." }
```


CHAPTER 07

Security Best Practices

Protect your API from common vulnerabilities and attack vectors.

1. ForceJson Middleware

ForceJsonResponse.php

```
class ForceJsonResponse { public function handle(Request $request, Closure $next): Response {  
    $request->headers->set('Accept', 'application/json'); return $next($request); } } // Register in  
bootstrap/app.php (Laravel 11+) ->withMiddleware(function (Middleware $middleware) {  
    $middleware->api(prepend: [ForceJsonResponse::class]); });
```

2. validated() vs all()

Bad — avoid this:

```
Post::create($request->all()); // could contain any field the client sends!
```

Good — do this instead:

```
Post::create($request->validated()); // only fields that passed validation
```

3. CORS — Explicit Origins Only

Bad — avoid this:

```
'allowed_origins' => ['*'], // never in production
```

Good — do this instead:

```
// config/cors.php 'allowed_origins' => ['https://myapp.com', 'https://admin.myapp.com'],
```

4. Token Rotation on Login

Delete old token before issuing a new one

```
$user->tokens()->where('name', $request->device)->delete(); $token =  
$user->createToken($request->device, ['posts:read']);
```

Tip: Enforce HTTPS at the infrastructure level. Never accept or serve API tokens over plain HTTP.

CHAPTER 08

Performance Optimisation

Build APIs that scale — avoid N+1, use caching, and paginate everything.

Eliminate N+1 Queries

The N+1 problem is the most common API performance killer. Always eager-load relationships and use Laravel Telescope to catch stray queries in development.

Bad — avoid this:

```
// 1 query for posts + 1 PER POST for the author = 16 queries for 15 posts! public function index() {  
    return PostResource::collection(Post::paginate(15)); // In Resource: $this->user->name => fires a  
    query per post! }
```

Good — do this instead:

```
// 2 queries total – posts + all their authors in one go public function index() { $posts =  
    Post::with(['user', 'tags', 'category']) ->withCount('comments') ->latest() ->paginate(15); return  
    PostResource::collection($posts); } // In Resource – safe even if not loaded: 'author' => new  
    UserResource($this->whenLoaded('user')),
```

Caching with Tag-based Invalidation

Cache::tags() pattern

```
public function index(Request $request) { $page = $request->get('page', 1); $posts =  
    Cache::tags(['posts'])->remember( "posts:page:{$page}", 300, // 5 minutes fn() =>  
    Post::with(['user', 'tags'])->latest()->paginate(15) ); return PostResource::collection($posts); }  
// Bust only posts cache when a post changes – not everything Cache::tags(['posts'])->flush();
```

Always Paginate

Bad — avoid this:

```
return Post::all(); // could return 50,000 rows!
```

Good — do this instead:

```
return Post::paginate(15); // always bounded return Post::cursorPaginate(20); // great for real-time  
feeds
```

Tip: Queue all heavy operations — emails, PDF generation, third-party API calls. Dispatch jobs and return 202 Accepted so requests stay fast.

CHAPTER 09

Testing API Endpoints

Write comprehensive feature tests — every endpoint, every edge case.

Test Three Scenarios for Every Endpoint

For each endpoint: (1) happy path, (2) validation failure, (3) authorisation failure.

PostControllerTest.php

```
class PostControllerTest extends TestCase { use RefreshDatabase; // 1. Happy path public function
test_user_can_create_post(): void { $token = User::factory()->create() ->createToken('test',
['posts:write'])->plainTextToken; $this->withToken($token) ->postJson('/api/v1/posts', [ 'title' =>
'Hello World', 'body' => 'This is the body content.', ]) ->assertCreated()
->assertJsonPath('data.title', 'Hello World'); $this->assertDatabaseHas('posts', ['title' => 'Hello
World']); } // 2. Validation failure public function test_title_is_required(): void { $token =
User::factory()->create() ->createToken('test', ['posts:write'])->plainTextToken;
$this->withToken($token) ->postJson('/api/v1/posts', ['body' => 'No title']) ->assertUnprocessable()
->assertJsonValidationErrors(['title']); } // 3. Authorisation failure public function
test_user_cannot_delete_anothers_post(): void { $post = Post::factory()->create(); // owned by a
different user $token = User::factory()->create() ->createToken('test',
['posts:write'])->plainTextToken; $this->withToken($token) ->deleteJson("/api/v1/posts/{$post->id}")
->assertForbidden(); } // 4. Unauthenticated public function
test_unauthenticated_cannot_create_post(): void { $this->postJson('/api/v1/posts', ['title' =>
'Test']) ->assertUnauthorized(); } }
```

Tip: Use `Http::fake()` to mock external HTTP calls, `Queue::fake()` to assert jobs were dispatched, and `Mail::fake()` to verify emails — all without real services.

CHAPTER 10

API Versioning

Version your API from day one — breaking changes are inevitable.

URI Versioning — The Explicit Approach

Prefix everything with `/v1/` so you can evolve the API without breaking existing clients. Clients using `/v1/posts` are never affected when you launch `/v2/posts`.

routes/api.php

```
// V1 – stable, maintained, never broken Route::prefix('v1')->name('v1.')->group(function () {
Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () { Route::apiResource('posts',
V1\PostController::class); Route::apiResource('users', V1\UserController::class); }); }); // V2 –
new features, evolved response shape Route::prefix('v2')->name('v2.')->group(function () {
Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () { Route::apiResource('posts',
V2\PostController::class); Route::apiResource('reactions', V2\ReactionController::class); }); });
```

Extend V1 in V2 — Only Override What Changed

V2/PostController.php

```
// V2 extends V1 – reuses all unchanged methods namespace App\Http\Controllers\Api\V2; class
PostController extends \App\Http\Controllers\Api\V1\PostController { // Only override the show
method which has a new V2 response shape public function show(Post $post): PostResource { return new
\App\Http\Resources\V2\PostResource( $post->load(['user', 'reactions'])) ); } }
```

Tip: Keep Resource classes separate per version (V1/PostResource, V2/PostResource). Never share resources between versions to prevent accidental coupling.

CHAPTER 11

Rate Limiting

Protect your API from abuse with layered, dynamic rate limits.

Multi-Tier Rate Limiting

Implement rate limits at multiple levels: per-second for burst protection, per-minute for normal use, per-plan for business tiers.

AppServiceProvider.php

```
public function boot(): void { // General API limiter RateLimiter::for('api', function (Request
$request) { $user = $request->user(); return [ // Burst protection: max 10 requests per second
Limit::perSecond(10)->by($user?->id ?: $request->ip()), // Per-plan limit (default 60/min for
guests) Limit::perMinute($user?->rate_limit ?? 60) ->by($user?->id ?: $request->ip()), ]; }); //
Strict auth limiter – stops brute-force login attacks RateLimiter::for('auth', function (Request
$request) { return Limit::perMinute(5)->by($request->ip()) ->response(fn() => response()->json([
'error' => 'too_many_attempts', 'message' => 'Too many login attempts. Try again in 1 minute.', ],
429)); }); // Upload limiter – expensive operations get stricter limits RateLimiter::for('uploads',
function (Request $request) { return Limit::perHour(20)->by($request->user()?->id ?:
$request->ip()); }); } // Apply in routes Route::middleware(['auth:sanctum',
'throttle:api'])->group(fn() => ...); Route::middleware('throttle:auth')->post('/login', ...);
Route::middleware(['auth:sanctum', 'throttle:uploads'])->post('/upload', ...);
```

Tip: Laravel automatically adds X-RateLimit-Limit, X-RateLimit-Remaining, and Retry-After headers when throttled. Document these so clients can backoff properly.

CHAPTER 12

API Documentation

Great APIs have great documentation — auto-generate it from your code.

Auto-Generate with Scribe

Scribe reads your controllers, Form Requests, and docblocks to generate full documentation automatically — no annotation clutter required.

Install and generate

```
composer require --dev knuckleswtf/scribe php artisan vendor:publish --tag=scribe-config php artisan scribe:generate # docs appear at /docs automatically
```

Add Docblocks to Guide Scribe

PostController.php — documented endpoint

```
/** * Create a post * * Creates a new blog post for the authenticated user. * Requires the 'posts:write' token ability. * * @authenticated * @response 201 { * "data": { "id": 1, "title": "Hello World", "status": "draft" } * } * @response 422 { * "error": "validation_failed", * "errors": { "title": ["The title field is required."] } * } */ public function store(StorePostRequest $request) { $post = $this->postService->create($request->validated()); return new PostResource($post); }
```

Documentation Tool Comparison

Scribe	Auto-generates from controllers & Form Requests	Most projects — zero-config start
Scramble	Static PHP analysis — no annotations needed	Avoiding annotation clutter
L5-Swagger	OpenAPI 3.0 docblock annotations	Teams needing full OpenAPI spec
Postman	Manual or exported via Scribe	Dev & QA team sharing

Quick Reference Cheatsheet

The most important rules at a glance.

Structure

- + Controllers only handle HTTP — move logic to Services/Actions
- + Separate API controllers under `Api\V1\` namespace
- + Use Form Request classes for all validation
- + Use API Resources for all responses — never raw models

Security

- + Enforce HTTPS at infrastructure level
- + Add `ForceJsonResponse` middleware to API group
- + Use `$request->validated()` — never `$request->all()`
- + Issue tokens with minimum required abilities
- + Rotate tokens on each login — delete old device tokens
- + Configure CORS with explicit allowed origins only

Performance

- + Always eager-load relationships with `with([])`
- + Always paginate list endpoints — never unbounded results
- + Cache expensive list queries with `Cache::tags()`
- + Select only required columns for list views
- + Queue all heavy operations — emails, PDFs, external APIs

Testing

- + Test every endpoint: success, validation, 401, 403, 404
- + Use `RefreshDatabase` trait in all feature tests
- + Mock external HTTP calls with `Http::fake()`
- + Assert exact JSON structure with `assertJsonStructure()`

API Design

- + Use HTTP verbs correctly (GET / POST / PUT / PATCH / DELETE)
- + Return 201 Created for POST, 204 No Content for DELETE
- + Version from day one — `/api/v1/` prefix on all routes
- + Use consistent error envelope: `{error, message, errors}`
- + Add rate limiting to all endpoints — stricter on auth routes